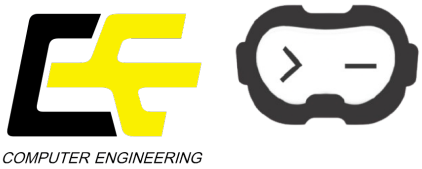


Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler & Dasar Pemrograman

Alvis Shohan Fawwaz Nizar - 5024241019

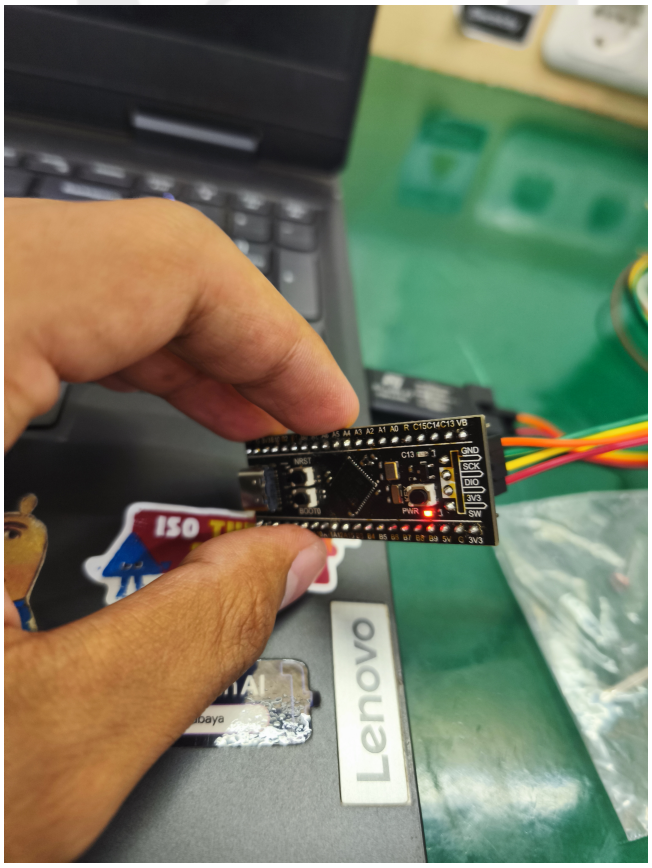
2026

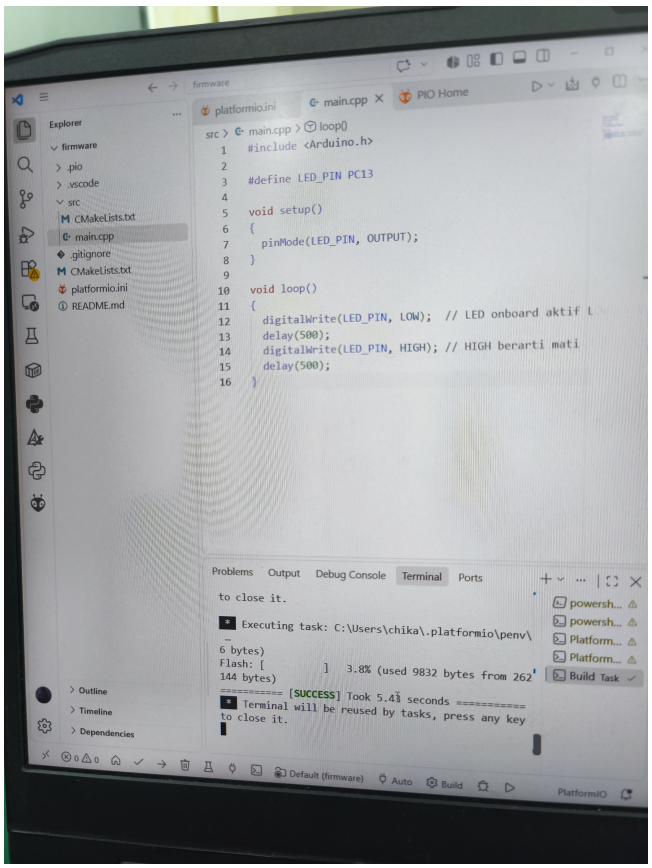
1 Langkah-Langkah Percobaan

1.1 PERCOBAAN 1: Pengenalan STM32 Blackpill (Blink)

Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan board STM32 Blackpill, ST-Link V2, kabel jumper, dan laptop yang telah terpasang Visual Studio Code serta PlatformIO.
2. Membuat project PlatformIO dan mengatur konfigurasi board Blackpill F401CC dengan framework Arduino pada file `platformio.ini`.
3. Menghubungkan pin SWDIO, SWCLK, GND, dan 3.3V pada ST-Link ke pin yang sesuai pada board STM32 Blackpill.
4. Menghubungkan ST-Link ke laptop melalui kabel USB.
5. Membuat program Blink pada file `src/main.cpp` dengan mengatur pin PC13 sebagai output digital.
6. Mengatur program agar LED menyala selama 500 ms dan mati selama 500 ms secara berulang.
7. Melakukan proses Build dan Upload program menggunakan PlatformIO.
8. Mengamati LED onboard STM32 Blackpill untuk memastikan program berjalan.

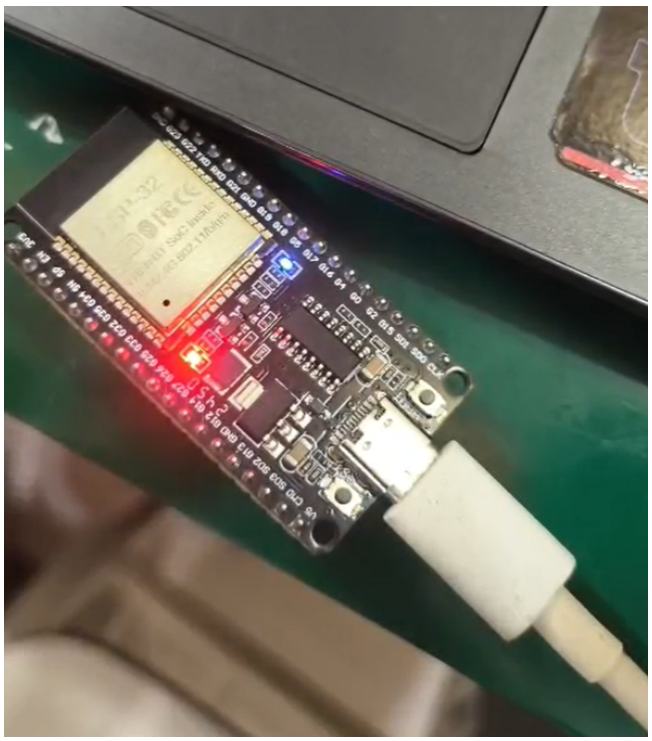


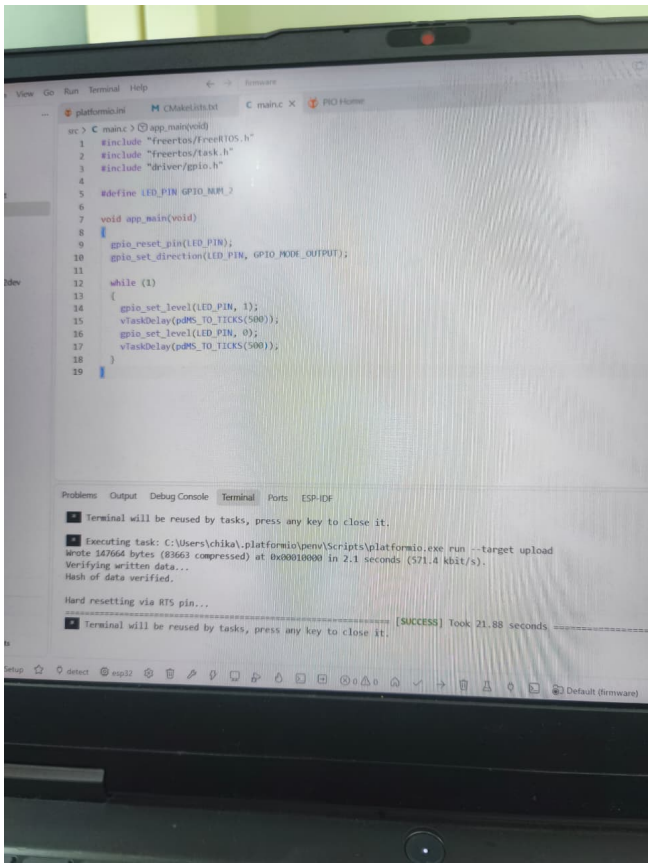


1.2 PERCOBAAN 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan board ESP32 DevKit dan kabel USB.
2. Mengubah konfigurasi project PlatformIO dengan memilih board ESP32 DevKit dan framework ESP-IDF pada file `platformio.ini`.
3. Menghubungkan ESP32 ke laptop menggunakan kabel USB.
4. Menghapus file `src/main.cpp` dan membuat file baru bernama `src/main.c`.
5. Menuliskan program Blink menggunakan framework ESP-IDF dengan fungsi `app_main()`, konfigurasi GPIO 2 sebagai output, dan fungsi `gpio_set_level()` untuk mengatur kondisi LED.
6. Mengatur jeda perubahan kondisi LED menggunakan fungsi `vTaskDelay()` selama 500 ms.
7. Melakukan proses Build dan Upload program menggunakan PlatformIO.
8. Mengamati LED onboard ESP32 untuk memastikan program Blink berhasil dijalankan.



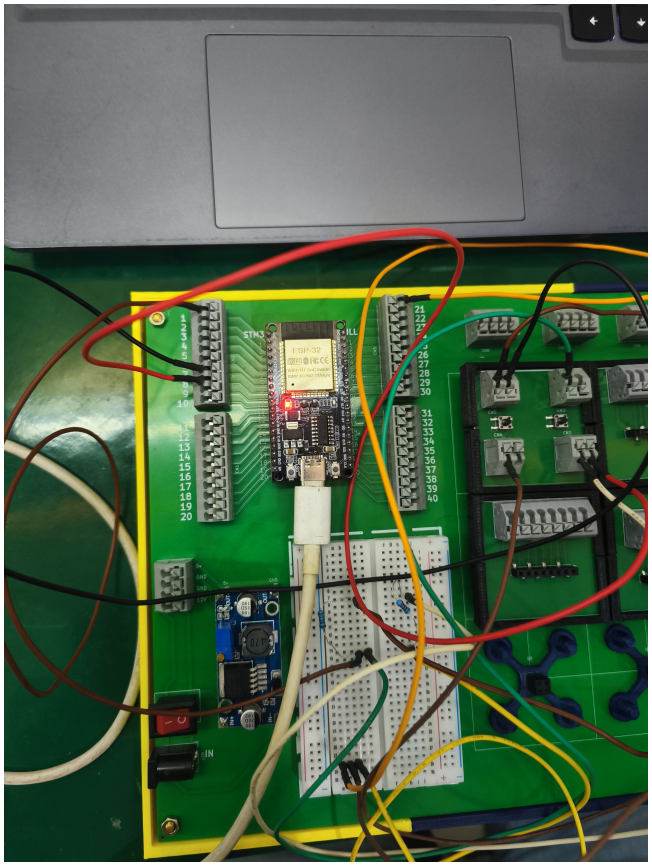


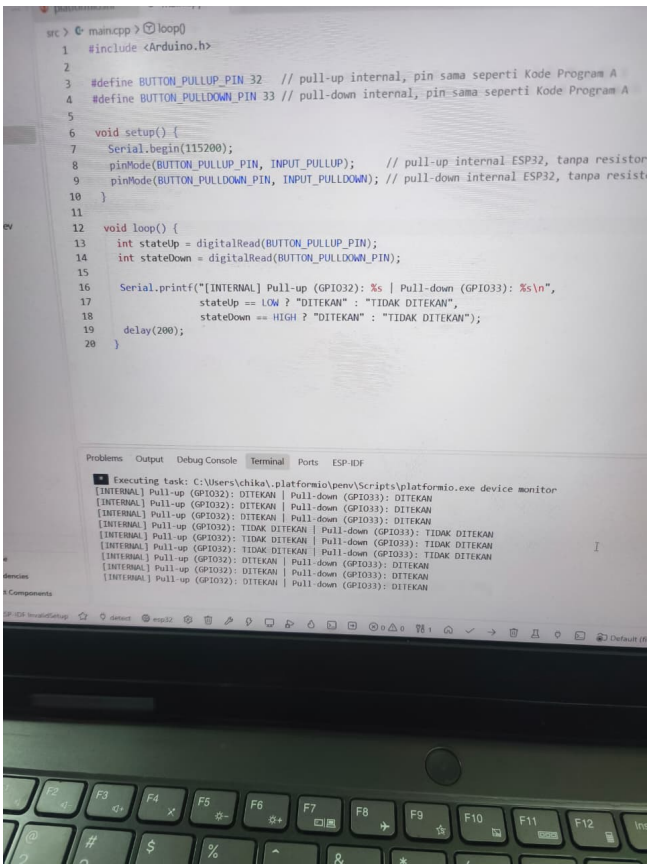
1.3 PERCOBAAN 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan board ESP32, dua pushbutton, dua resistor 10 kΩ, breadboard, dan kabel jumper.

2. Mengatur konfigurasi PlatformIO menggunakan board ESP32 DevKit dengan framework Arduino.
3. Merangkai pushbutton pertama pada GPIO 32 menggunakan konfigurasi pull-up eksternal dengan resistor 10 kΩ yang terhubung ke 3.3V dan tombol yang terhubung ke GND.
4. Merangkai pushbutton kedua pada GPIO 33 menggunakan konfigurasi pull-down eksternal dengan resistor 10 kΩ yang terhubung ke GND dan tombol yang terhubung ke 3.3V.
5. Menuliskan program untuk membaca kondisi kedua tombol menggunakan fungsi `digitalRead()` dengan konfigurasi pin sebagai input biasa.
6. Melakukan proses Build dan Upload program, kemudian membuka Serial Monitor dengan baud rate 115200.
7. Menekan dan melepaskan kedua tombol secara bergantian untuk mengamati perubahan kondisi logika pada masing-masing pin.
8. Melepas kedua resistor eksternal tanpa mengubah posisi pushbutton pada rangkaian.
9. Mengubah konfigurasi GPIO 32 menjadi `INPUT_PULLUP` dan GPIO 33 menjadi `INPUT_PULLDOWN` pada program.
10. Melakukan proses Build dan Upload kembali, kemudian menguji kedua tombol menggunakan Serial Monitor.
11. Membandingkan hasil pembacaan tombol pada konfigurasi pull-up dan pull-down eksternal dengan konfigurasi internal.



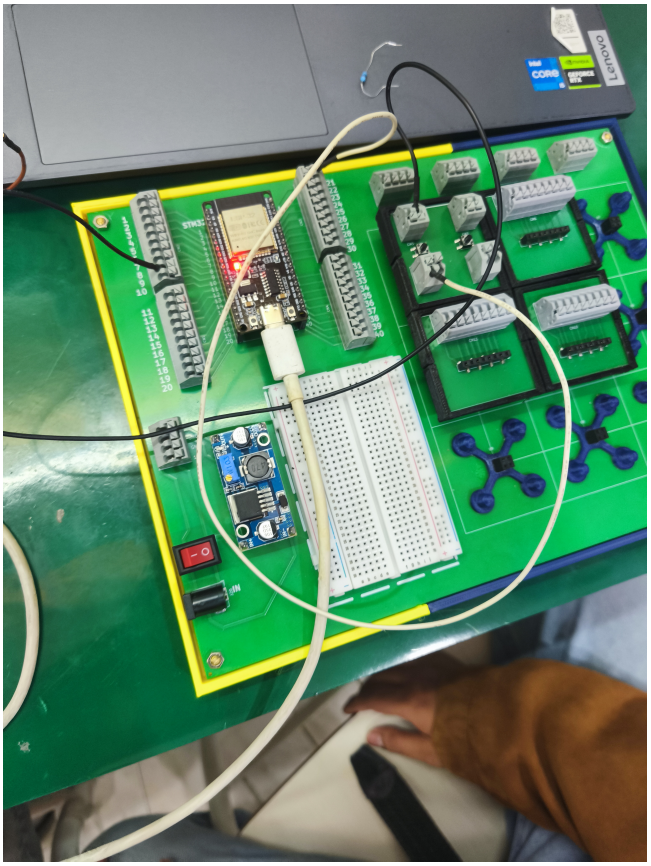


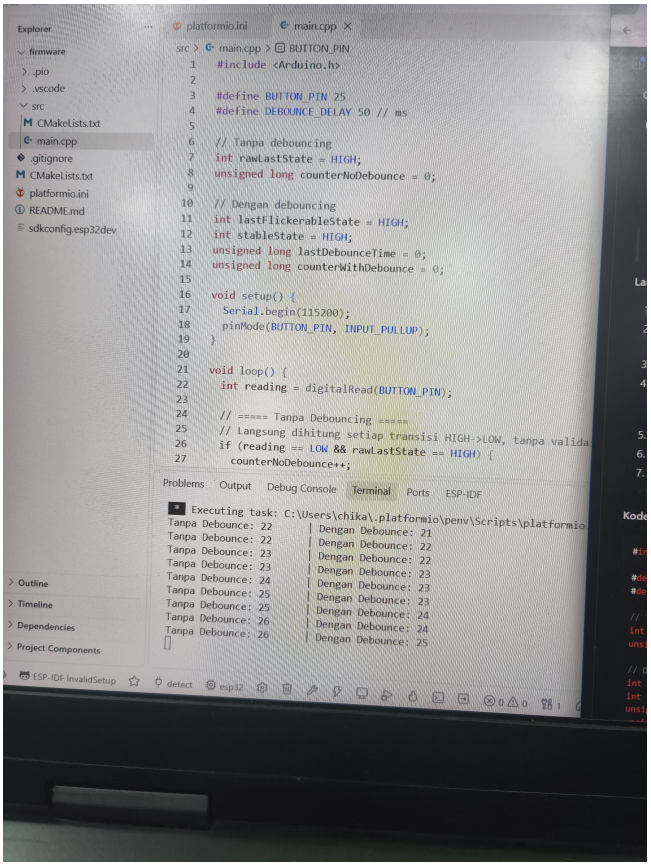
1.4 PERCOBAAN 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan board ESP32, pushbutton, breadboard, dan kabel jumper.
2. Menghubungkan pushbutton ke GPIO 25 dan GND pada ESP32.
3. Mengatur konfigurasi pin GPIO 25 sebagai input menggunakan `INPUT_PULLUP`.
4. Menuliskan program untuk membandingkan pembacaan tombol tanpa debouncing dan dengan debouncing menggunakan dua variabel penghitung.
5. Mengatur nilai awal `DEBOUNCE_DELAY` sebesar 50 ms.
6. Melakukan proses Build dan Upload program, kemudian membuka Serial Monitor.
7. Menekan tombol beberapa kali dan mengamati perbedaan nilai penghitung tanpa debouncing dan dengan debouncing.
8. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload kembali.
9. Mengulangi pengujian dengan menekan tombol dan mengamati perubahan kedua penghitung.
10. Mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian melakukan Build dan Upload kembali.

11. Mengulangi pengujian dan membandingkan hasil pembacaan tombol pada waktu debounce 10 ms, 50 ms, dan 200 ms.

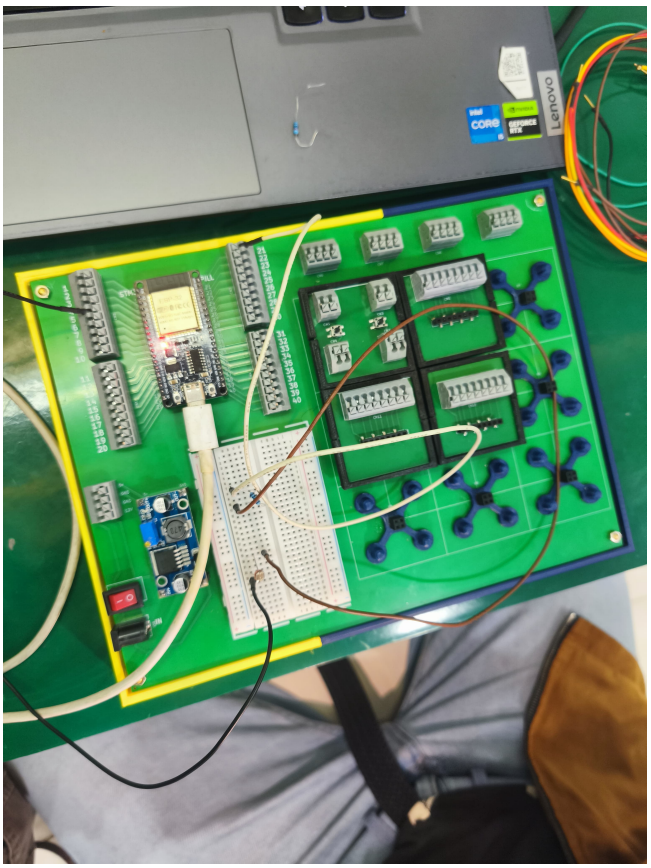


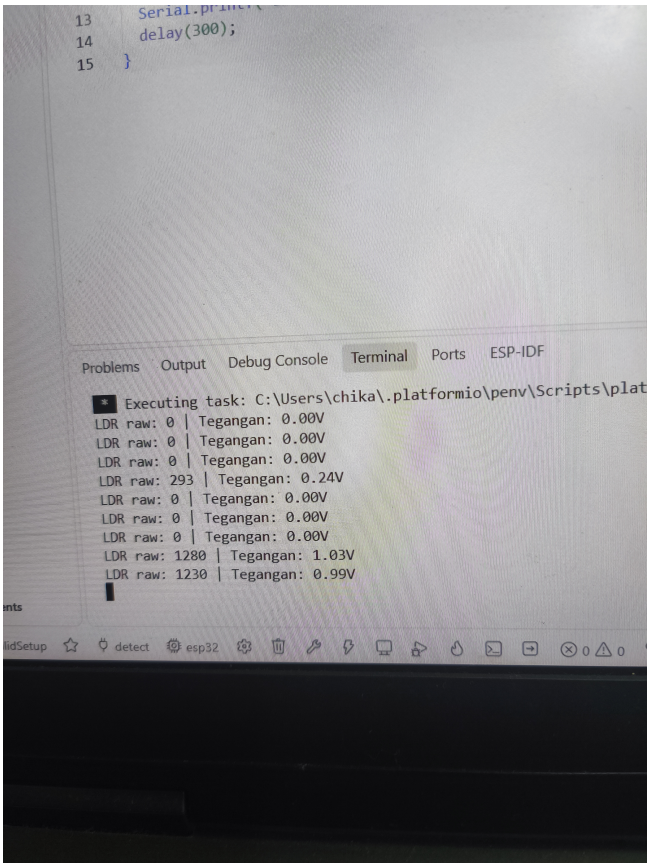


1.5 PERCOBAAN 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan board ESP32, sensor LDR, resistor 10 kΩ, breadboard, dan kabel jumper.
2. Mengatur konfigurasi PlatformIO menggunakan board ESP32 DevKit dengan framework Arduino.
3. Merangkai LDR dan resistor 10 kΩ sebagai pembagi tegangan, dengan salah satu kaki LDR terhubung ke 3.3V dan kaki lainnya terhubung ke GPIO 34.
4. Menghubungkan titik pertemuan antara LDR dan resistor ke GPIO 34 serta menghubungkan resistor ke GND.
5. Menuliskan program untuk membaca nilai analog menggunakan fungsi `analogRead()` pada GPIO 34.
6. Menambahkan perhitungan konversi nilai ADC menjadi tegangan menggunakan resolusi 12-bit dengan rentang 0 hingga 4095.
7. Melakukan proses Build dan Upload program menggunakan PlatformIO.
8. Membuka Serial Monitor dengan baud rate 115200 untuk mengamati nilai ADC dan tegangan keluaran sensor.
9. Menutup permukaan LDR menggunakan tangan untuk mengurangi intensitas cahaya yang diterima sensor.
10. Memberikan cahaya pada LDR menggunakan lampu dan mengamati perubahan nilai ADC serta tegangan yang ditampilkan pada Serial Monitor.
11. Membandingkan hasil pembacaan sensor pada kondisi cahaya yang berbeda.




delay(300);
}
Problems
Output
Debug Console
Terminal
Ports
ESP-IDF
Executing task: C:\Users\chika\.platformio\penv\Scripts\plat
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 293 | Tegangan: 0.24V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 1280 | Tegangan: 1.03V
LDR raw: 1230 | Tegangan: 0.99V
detect
esp32

2 Analisis Hasil Percobaan

Berdasarkan hasil praktikum yang telah dilakukan, percobaan pertama menggunakan board STM32 Blackpill berhasil dijalankan dengan baik. Program Blink yang dibuat menggunakan framework Arduino berhasil diunggah melalui ST-Link dan menghasilkan perubahan kondisi LED onboard yang terhubung pada pin PC13 secara berulang dengan interval waktu 500 ms. Hasil tersebut menunjukkan bahwa konfigurasi PlatformIO, komunikasi antara ST-Link dengan board, serta pengaturan GPIO sebagai output digital telah berjalan sesuai dengan yang diharapkan. LED pada STM32 Blackpill menggunakan logika aktif LOW, sehingga LED menyala ketika pin berada pada kondisi LOW dan mati ketika berada pada kondisi HIGH.

Pada percobaan kedua, program Blink berhasil dijalankan pada board ESP32 menggunakan framework ESP-IDF. LED onboard ESP32 dapat menyala dan mati secara berulang sesuai interval waktu yang telah ditentukan. Program menggunakan fungsi `app_main()` sebagai titik masuk program, kemudian mengatur GPIO 2 sebagai output menggunakan fungsi `gpio_set_direction()` dan mengubah kondisi keluarannya menggunakan fungsi `gpio_set_level()`. Pengaturan jeda dilakukan menggunakan fungsi `vTaskDelay()`. Hasil tersebut menunjukkan bahwa ESP32 dapat dikendalikan menggunakan framework ESP-IDF. Berbeda dengan framework Arduino yang menggunakan struktur `setup()` dan `loop()`, ESP-IDF menggunakan struktur program yang terintegrasi dengan FreeRTOS.

Pada percobaan ketiga, pengujian pull-up dan pull-down eksternal maupun internal berhasil dilakukan menggunakan dua pushbutton pada ESP32. Hasil pembacaan menunjukkan bahwa pada konfigurasi pull-up, kondisi tombol tidak ditekan menghasilkan logika HIGH, sedangkan ketika ditekan menghasilkan logika LOW. Sebaliknya, pada konfigurasi pull-down, kondisi tombol tidak ditekan menghasilkan logika LOW dan ketika ditekan menghasilkan logika HIGH. Penggunaan resistor internal juga memberikan hasil pembacaan yang sesuai dengan konfigurasi eksternal. Hal ini menunjukkan bahwa kedua metode dapat digunakan untuk menjaga kondisi logika pin GPIO agar tidak mengalami floating ketika tombol tidak ditekan. Perbedaan utama keduanya terletak pada penggunaan komponen, karena konfigurasi internal memanfaatkan resistor yang telah tersedia di dalam mikrokontroler sehingga tidak memerlukan resistor tambahan pada rangkaian.

Pada percobaan keempat, pengujian debouncing berhasil dilakukan untuk membandingkan pembacaan tombol tanpa debouncing dan dengan debouncing. Pembacaan tanpa debouncing berpotensi mendeteksi beberapa perubahan sinyal dalam waktu singkat akibat bouncing pada kontak mekanis pushbutton. Kondisi tersebut dapat menyebabkan satu kali penekanan tombol terbaca sebagai lebih dari satu perubahan. Sementara itu, penerapan debouncing menggunakan fungsi `millis()` memungkinkan program menunggu hingga kondisi tombol stabil selama waktu tertentu sebelum perubahan tersebut dihitung sebagai penekanan yang valid. Pengujian dilakukan menggunakan variasi waktu debounce sebesar 10 ms, 50 ms, dan 200 ms. Perbedaan waktu tersebut memengaruhi respons pembacaan tombol, karena waktu debounce yang terlalu singkat berpotensi belum mampu mengatasi seluruh bouncing, sedangkan waktu yang terlalu lama dapat memperlambat respons terhadap penekanan berikutnya. Dengan demikian, penggunaan debouncing dapat membantu menghasilkan pembacaan tombol yang lebih stabil dan mengurangi kesalahan penghitungan.

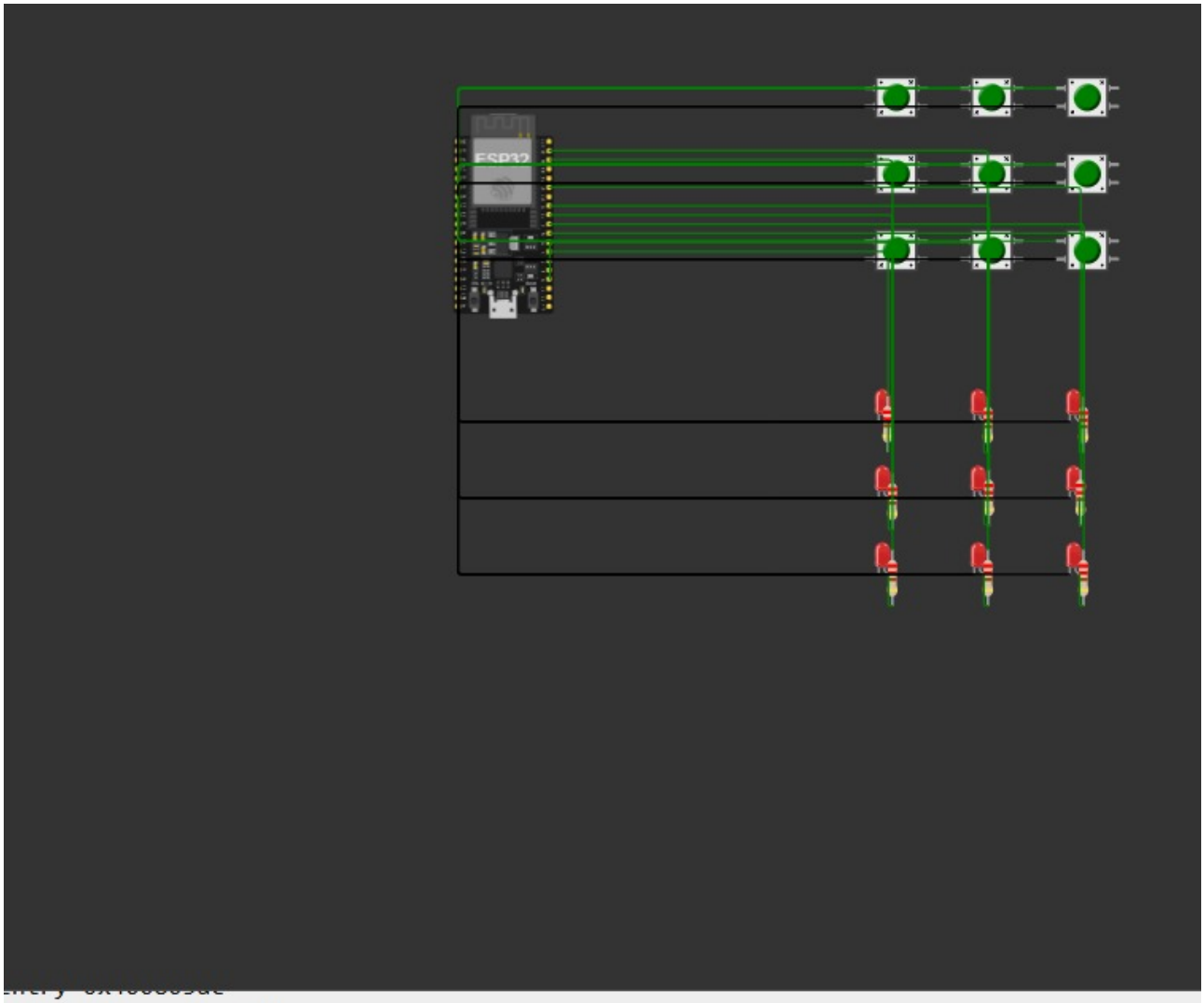
Pada percobaan kelima, pembacaan nilai analog menggunakan sensor LDR berhasil dilakukan melalui ADC pada GPIO 34 ESP32. Hasil pengujian menunjukkan bahwa perubahan intensitas cahaya yang diterima LDR menyebabkan perubahan nilai resistansi dan tegangan keluaran pada rangkaian pembagi tegangan. Ketika intensitas cahaya meningkat, resistansi LDR menurun sehingga

tegangan pada titik tengah rangkaian meningkat dan menghasilkan nilai ADC yang lebih besar. Sebaliknya, ketika intensitas cahaya berkurang, resistansi LDR meningkat sehingga tegangan keluaran dan nilai ADC cenderung menurun. Nilai ADC yang terbaca berada pada rentang 0 hingga 4095 karena menggunakan resolusi 12-bit. Nilai tersebut kemudian dikonversi menjadi perkiraan tegangan untuk mempermudah pengamatan perubahan keluaran sensor. Hasil ini menunjukkan bahwa ESP32 mampu membaca perubahan besaran analog dari lingkungan melalui sensor LDR.

Secara keseluruhan, seluruh percobaan pada Modul 1 berhasil dilaksanakan. Pengujian Blink pada STM32 Blackpill dan ESP32 menunjukkan bahwa mikrokontroler dapat mengendalikan keluaran digital menggunakan framework yang berbeda. Pengujian pull-up dan pull-down menunjukkan pentingnya menjaga kestabilan kondisi logika pada pin input, sedangkan percobaan debouncing menunjukkan perlunya penyaringan sinyal untuk memperoleh pembacaan tombol yang lebih akurat. Selain itu, percobaan pembacaan ADC menggunakan LDR menunjukkan kemampuan ESP32 dalam membaca perubahan sinyal analog dari lingkungan. Dengan demikian, tujuan praktikum untuk memahami penggunaan GPIO sebagai input dan output digital serta analog pada sistem tertanam telah tercapai.

3 Hasil Tugas Modul

3.1 Tugas Modul Simulasi



```
Sistem Debouncing Siap
Tombol 1 DITEKAN
Tombol 1 DILEPAS
Tombol 1 DITEKAN
Tombol 1 DILEPAS
Tombol 3 DITEKAN
```

Listing 1: Program Debouncing Matrix Pushbutton ESP32

```
const int buttonPins[9] = {
    13, 12, 14,
    27, 26, 25,
    33, 32, 15
};

const int ledPins[9] = {
    4, 16, 17,
    18, 19, 21,
```

```
    22, 23, 5
};

const unsigned long debounceDelay = 100;

int stableState[9];
int lastReading[9];
unsigned long lastDebounceTime[9];

void setup() {
    Serial.begin(115200);

    for (int i = 0; i < 9; i++) {
        pinMode(buttonPins[i], INPUT_PULLUP);
        pinMode(ledPins[i], OUTPUT);

        stableState[i] = HIGH;
        lastReading[i] = HIGH;
        lastDebounceTime[i] = 0;

        digitalWrite(ledPins[i], LOW);
    }

    Serial.println("Sistem Debouncing Siap");
}

void loop() {
    for (int i = 0; i < 9; i++) {
        int reading = digitalRead(buttonPins[i]);

        if (reading != lastReading[i]) {
            lastDebounceTime[i] = millis();
        }

        if (millis() - lastDebounceTime[i] >= debounceDelay) {
            if (reading != stableState[i]) {
                stableState[i] = reading;

                if (stableState[i] == LOW) {
                    digitalWrite(ledPins[i], HIGH);

                    Serial.print("Tombol ");
                    Serial.print(i + 1);
                    Serial.println(" DITEKAN");
                } else {
                    digitalWrite(ledPins[i], LOW);

                    Serial.print("Tombol ");
                    Serial.print(i + 1);
                    Serial.println(" DILEPAS");
                }
            }
        }

```

```
64          lastReading[i] = reading;
65      }
66  }
```

3.2 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.

Debouncing tetap diperlukan karena kontak mekanis di dalam pushbutton tidak langsung stabil ketika ditekan atau dilepas. Kontak dapat mengalami pantulan (bouncing) sehingga menghasilkan perubahan sinyal HIGH dan LOW secara cepat dalam waktu singkat. Akibatnya, satu kali penekanan tombol dapat terbaca sebagai beberapa kali penekanan oleh ESP32. Debouncing digunakan untuk memastikan perubahan kondisi tombol stabil selama 100 ms sebelum diproses.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

Rangkaian menggunakan konfigurasi internal pull-up melalui `INPUT_PULLUP`. Ketika tombol tidak ditekan, pin GPIO berada pada kondisi HIGH karena terhubung secara internal ke tegangan 3,3 V. Ketika tombol ditekan, pin terhubung ke GND sehingga kondisinya menjadi LOW. Kondisi pin tidak boleh dibiarkan floating karena tegangan input menjadi tidak menentu dan dapat menyebabkan pembacaan HIGH atau LOW secara acak.

3. Andaikan salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

Digital input hanya membaca dua kondisi logika, yaitu HIGH dan LOW, sehingga sesuai untuk membaca tombol yang memiliki kondisi ditekan atau tidak ditekan. Sementara itu, analog input menggunakan ADC (*Analog to Digital Converter*) untuk mengubah tegangan analog menjadi nilai digital dalam rentang tertentu. Pada ESP32 dengan ADC 12-bit, nilai pembacaan berkisar antara 0–4095. Oleh karena itu, LDR dapat menghasilkan berbagai nilai pembacaan sesuai intensitas cahaya yang diterima, bukan hanya dua kondisi seperti tombol.

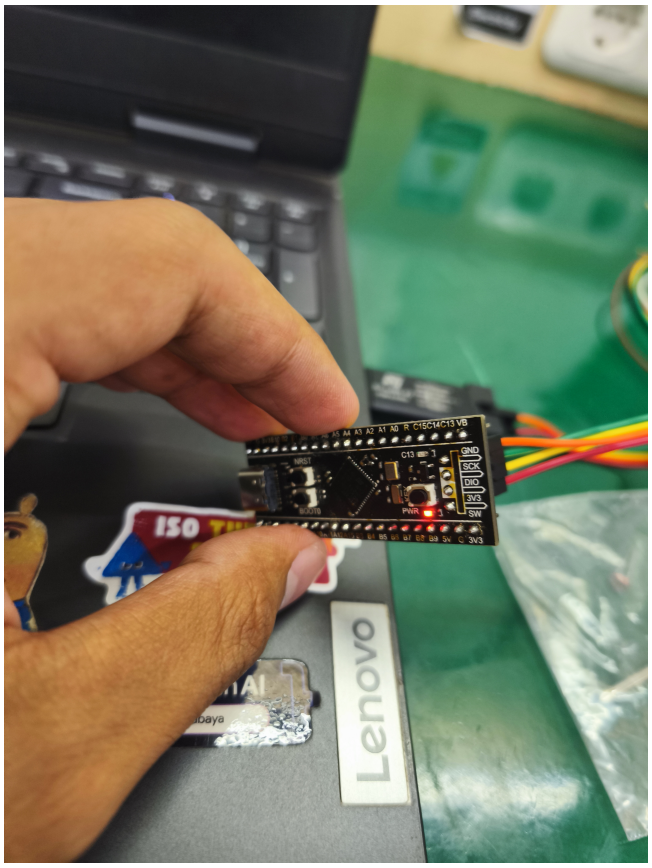
4 Kesimpulan

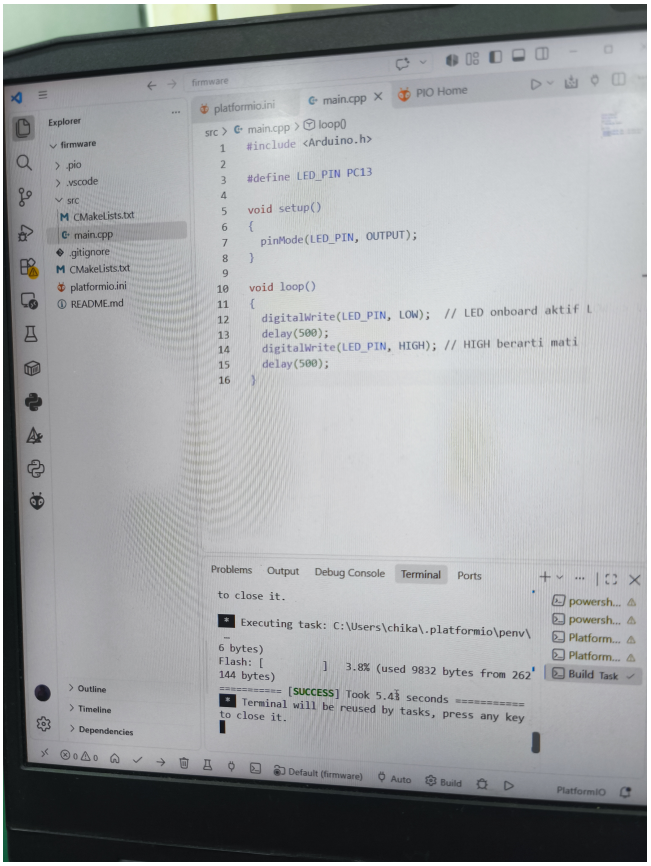
Berdasarkan praktikum Pengantar Mikrokontroler yang telah dilakukan, dapat disimpulkan bahwa mikrokontroler dapat digunakan untuk mengendalikan perangkat melalui konfigurasi dan pemrograman GPIO. Percobaan Blink pada STM32 Blackpill dan ESP32 menunjukkan penerapan keluaran digital untuk mengendalikan LED. Percobaan pull-up dan pull-down menunjukkan bahwa konfigurasi resistor diperlukan untuk menentukan kondisi logika pin ketika tombol tidak ditekan agar input tidak mengalami kondisi floating. Selain itu, percobaan debouncing menunjukkan pentingnya memberikan jeda untuk memastikan perubahan kondisi tombol telah stabil sebelum diproses oleh mikrokontroler. Pada percobaan ADC menggunakan LDR, diperoleh pemahaman bahwa mikrokontroler dapat membaca perubahan besaran analog dan mengubahnya menjadi nilai digital dengan resolusi 12-bit. Melalui

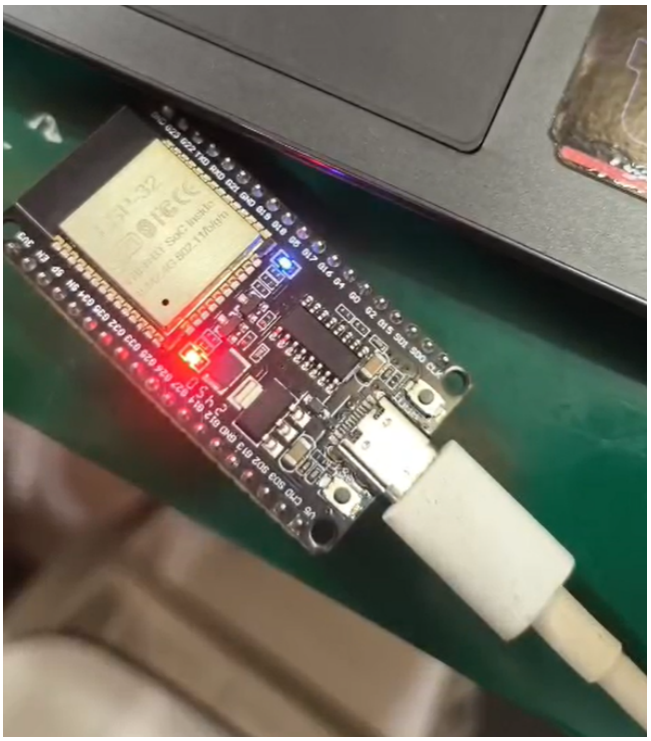
tugas modul, konsep input digital, debouncing, dan output LED diterapkan untuk membangun sistem kendali menggunakan pushbutton 3x3 dan LED matrix 3x3. Secara keseluruhan, praktikum ini memberikan pemahaman mengenai konfigurasi GPIO, pembacaan input digital dan analog, serta pengendalian perangkat menggunakan mikrokontroler ESP32.

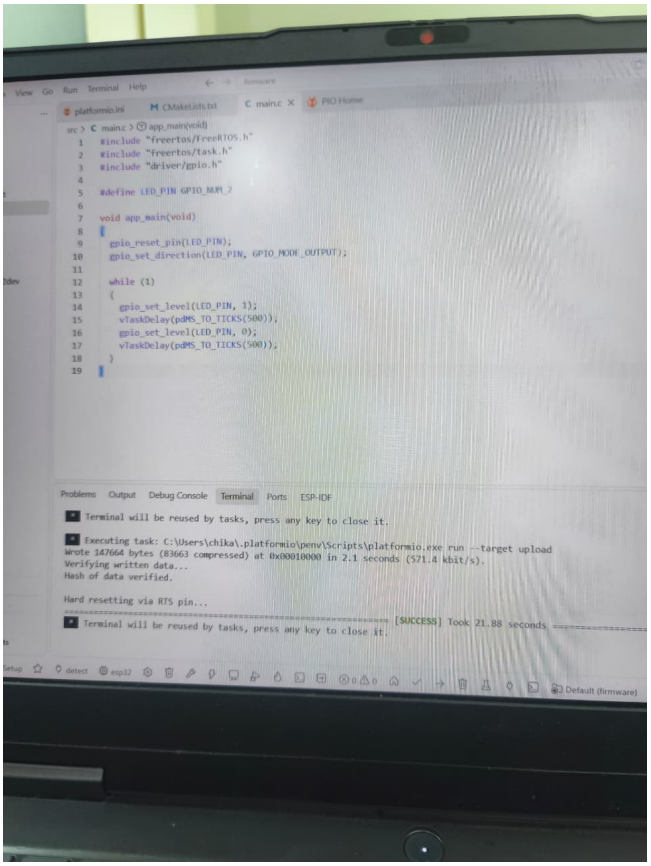
5 Lampiran

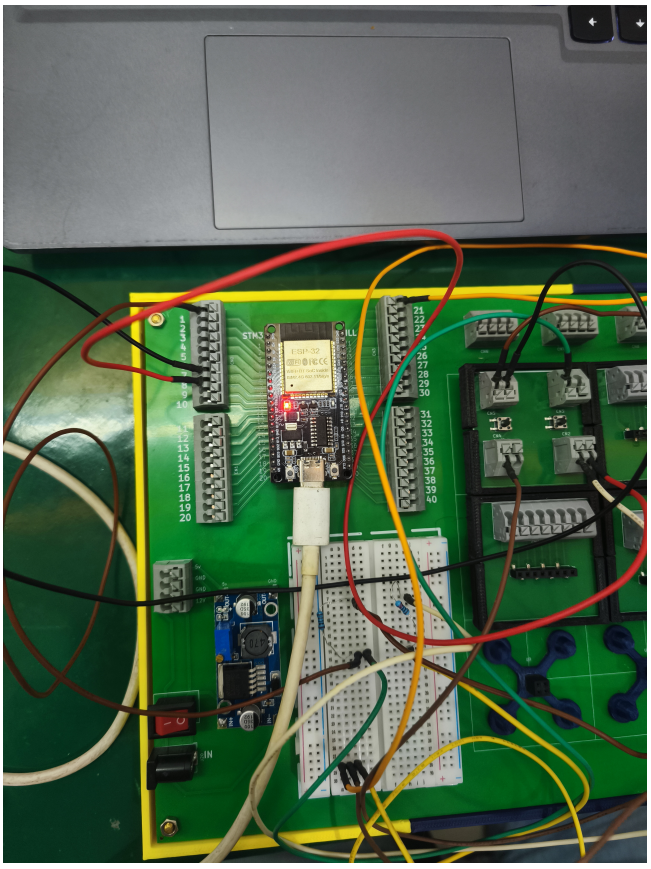
5.1 Dokumentasi saat praktikum

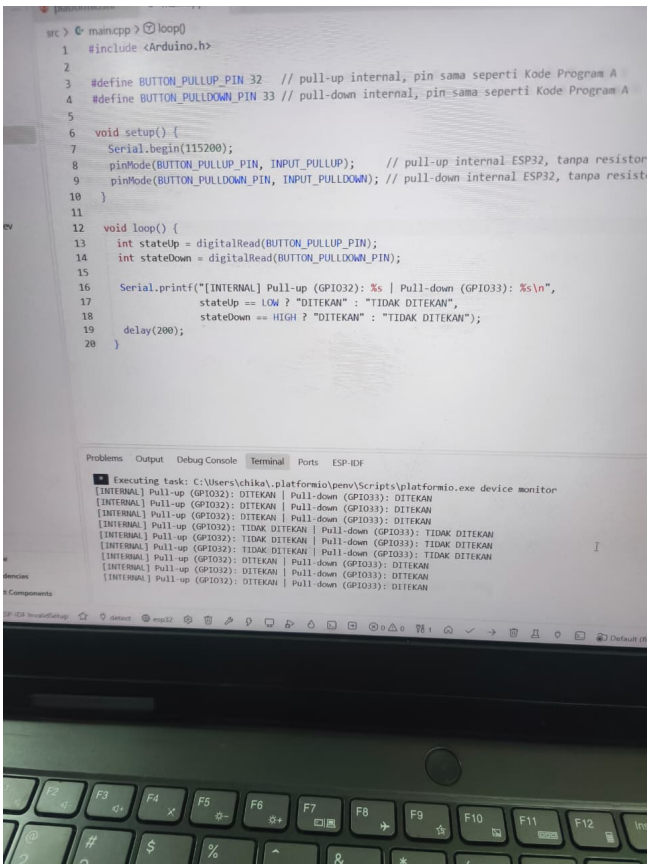


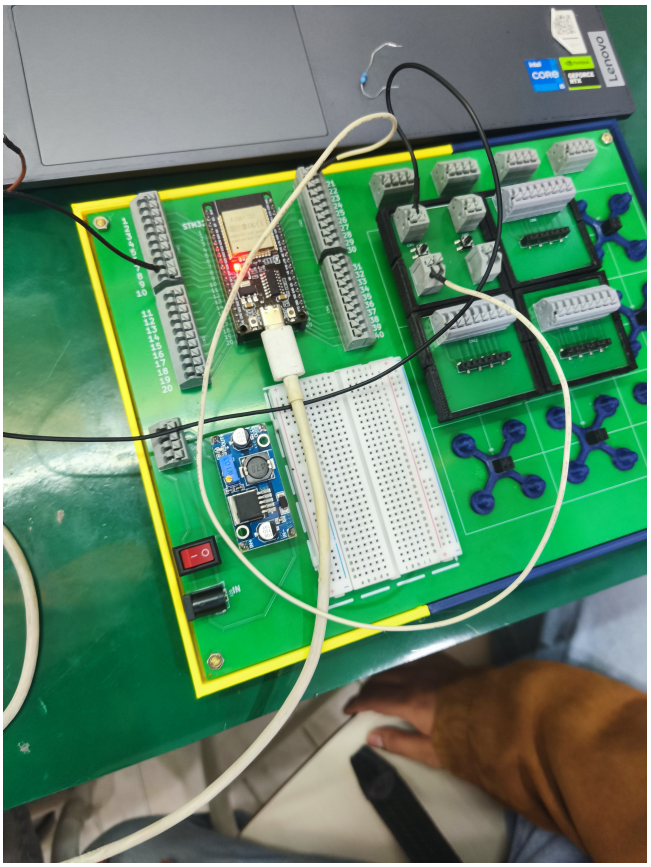

firmware
platformio.ini
main.cpp
PIO Home
Explorer
src
main.cpp
loop()
#include <Arduino.h>
#define LED_PIN PC13
void setup()
{
pinMode(LED_PIN, OUTPUT);
}
void loop()
{
digitalWrite(LED_PIN, LOW); // LED onboard aktif L
delay(500);
digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
delay(500);
}
.pio
.vscode
src
CMakeLists.txt
main.cpp
.gitignore
CMakeLists.txt
platformio.ini
README.md
Problems
Output
Debug Console
Terminal
Ports
to close it.
Executing task: C:\Users\chika\.platformio\penv\
6 bytes)
Flash: [ ] 3.8% (used 9832 bytes from 262
144 bytes)
========== [SUCCESS] Took 5.43 seconds ==========
Terminal will be reused by tasks, press any key
to close it.
powersh...
powersh...
Platform...
Platform...
Build Task
Outline
Timeline
Dependencies
Default (firmware)
Auto
Build
PlatformIO

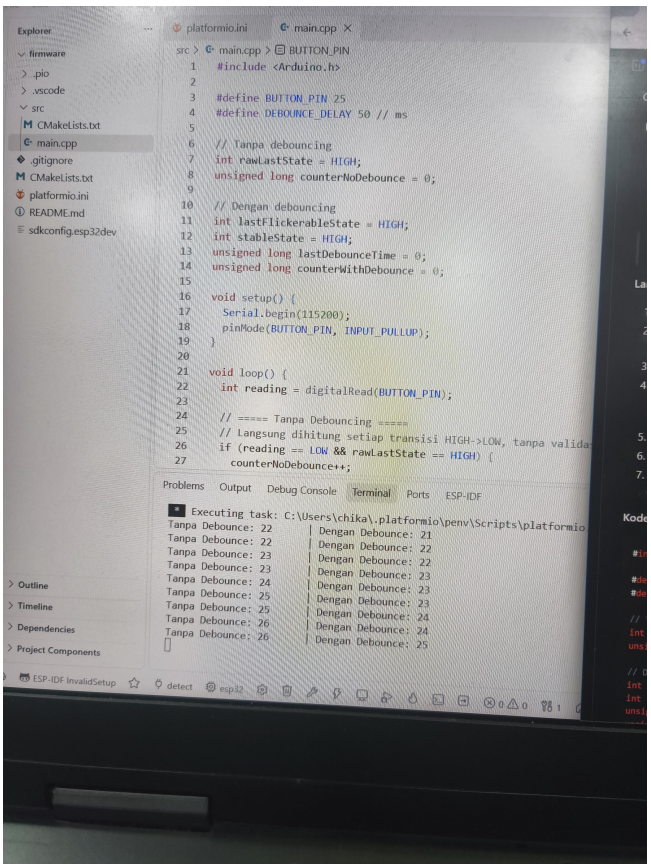


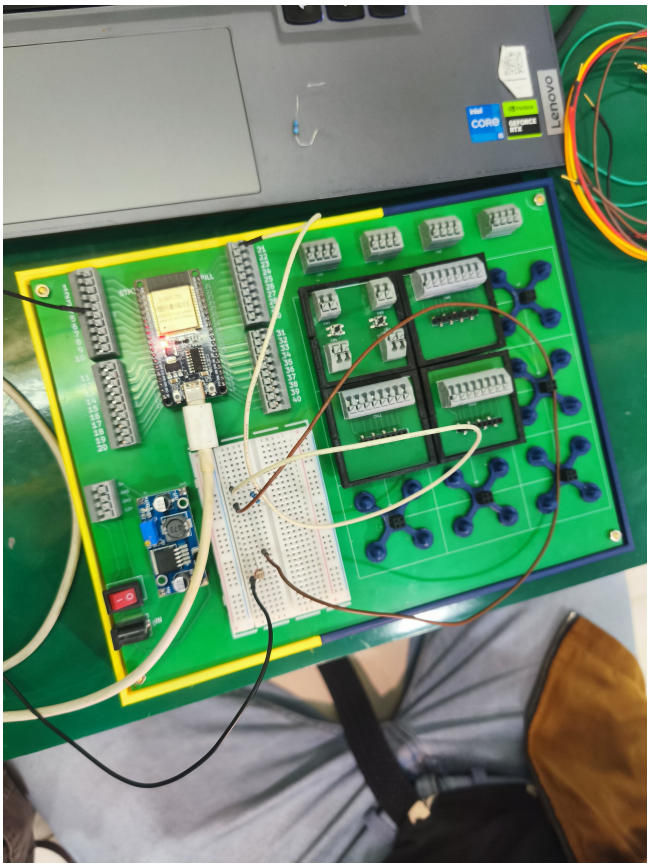

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#define LED_PIN GPIO_NUM_2
void app_main(void)
{
gpio_reset_pin(LED_PIN);
gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
while (1)
{
gpio_set_level(LED_PIN, 1);
vTaskDelay(pdMS_TO_TICKS(500));
gpio_set_level(LED_PIN, 0);
vTaskDelay(pdMS_TO_TICKS(500));
}
}
Problems
Output
Debug Console
Terminal
Ports
ESP-IDF
Terminal will be reused by tasks, press any key to close it.
Executing task: C:\Users\chika\.platformio\penv\Scripts\platformio.exe run --target upload
Wrote 147664 bytes (83663 compressed) at 0x00010000 in 2.1 seconds (571.4 kbit/s).
Verifying written data...
Hash of data verified.
Hard resetting via RTS pin...
[SUCCESS] Took 21.88 seconds
Terminal will be reused by tasks, press any key to close it.

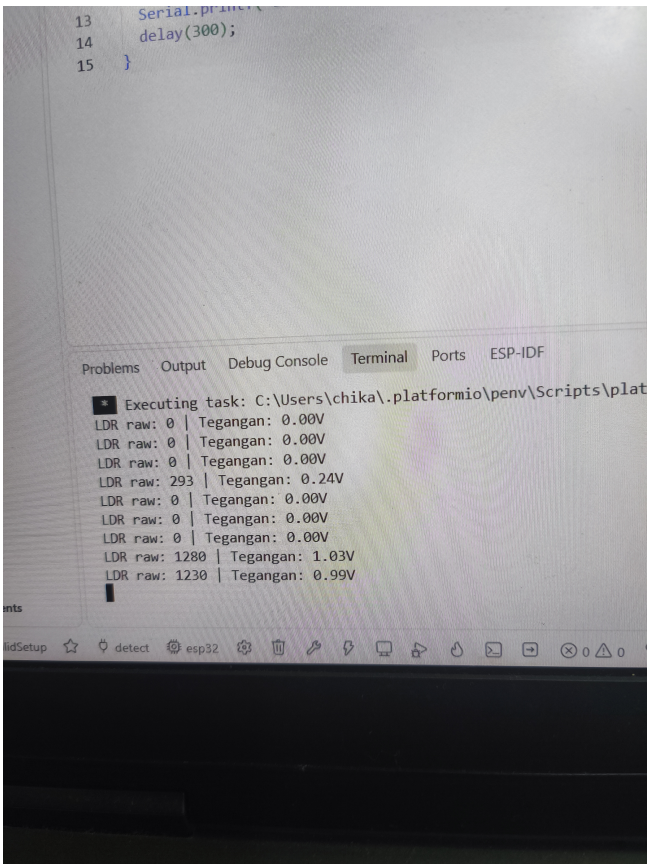



src > main.cpp > loop()
#include <Arduino.h>
#define BUTTON_PULLUP_PIN 32 // pull-up internal, pin sama seperti Kode Program A
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal, pin sama seperti Kode Program A
void setup() {
Serial.begin(115200);
pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP); // pull-up internal ESP32, tanpa resistor
pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32, tanpa resist
}
void loop() {
int stateUp = digitalRead(BUTTON_PULLUP_PIN);
int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);
Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
delay(200);
}
Problems Output Debug Console Terminal Ports ESP-IDF
Executing task: C:\Users\chika\.platformio\penv\Scripts\platformio.exe device monitor
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN
[INTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[INTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[INTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN
[INTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): DITEKAN




Explorer
firmware
.pio
.vscode
src
CMakeLists.txt
main.cpp
.gitignore
CMakeLists.txt
platformio.ini
README.md
sdkconfig.esp32dev
platformio.ini
main.cpp
src > main.cpp > BUTTON_PIN
#include <Arduino.h>
#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms
// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;
// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;
void setup() {
Serial.begin(115200);
pinMode(BUTTON_PIN, INPUT_PULLUP);
}
void loop() {
int reading = digitalRead(BUTTON_PIN);
// ===== Tanpa Debouncing =====
// Langsung dihitung setiap transisi HIGH->LOW, tanpa valida
if (reading == LOW && rawLastState == HIGH) {
counterNoDebounce++;
Problems
Output
Debug Console
Terminal
Ports
ESP-IDF
Executing task: C:\Users\chika\.platformio\penv\Scripts\platformio
Tanpa Debounce: 22 | Dengan Debounce: 21
Tanpa Debounce: 22 | Dengan Debounce: 22
Tanpa Debounce: 23 | Dengan Debounce: 22
Tanpa Debounce: 23 | Dengan Debounce: 23
Tanpa Debounce: 24 | Dengan Debounce: 23
Tanpa Debounce: 25 | Dengan Debounce: 23
Tanpa Debounce: 25 | Dengan Debounce: 24
Tanpa Debounce: 26 | Dengan Debounce: 24
Tanpa Debounce: 26 | Dengan Debounce: 25
Outline
Timeline
Dependencies
Project Components
ESP-IDF InvalidSetup
detect
esp32




13 Serial.print(
14 delay(300);
15 }
Problems
Output
Debug Console
Terminal
Ports
ESP-IDF
Executing task: C:\Users\chika\.platformio\penv\Scripts\plat
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 293 | Tegangan: 0.24V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 0 | Tegangan: 0.00V
LDR raw: 1280 | Tegangan: 1.03V
LDR raw: 1230 | Tegangan: 0.99V
detect
esp32